

ARQUITECTURA_MICROSERVICIOS

Arquitectura de microservicios — S.I.G.I

Integrantes: Hawk Durant, Emilio Jaramillo, Rodrigo Candia

Proyecto: Sistema Inteligente de Gestión de Incendios — Municipalidad Valle del Sol

Duoc UC — Ingeniería en Informática

1. Decisión arquitectónica

Optamos por **microservicios** porque el dominio del municipio tiene procesos bien separados: usuarios, reportes, emergencias, recursos, notificaciones, empleos y archivos multimedia. Cada uno puede escalar y desplegarse de forma independiente. Además, el enunciado del caso pedía explícitamente API REST, persistencia por servicio y comunicación entre componentes.

No usamos procedimientos almacenados en MySQL: toda la persistencia la resolvimos con **Spring Data JPA** y entidades anotadas, lo que encaja mejor con el estilo Spring Boot y facilita las pruebas unitarias con mocks.

2. Diagrama de arquitectura

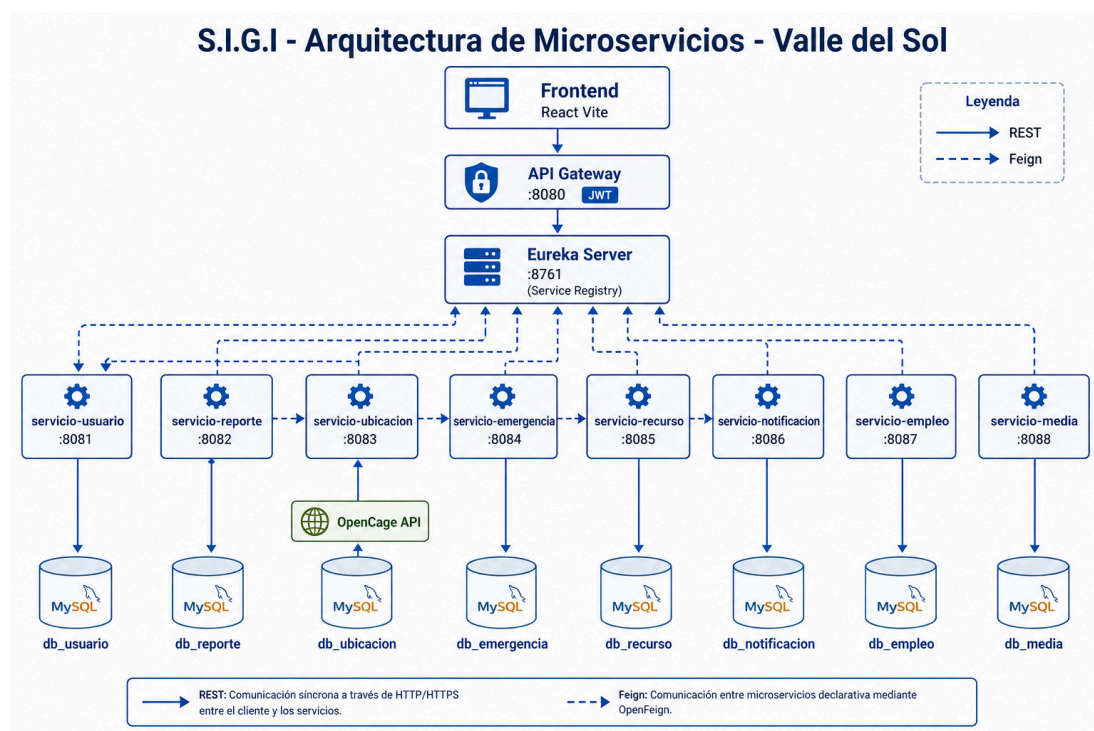


Diagrama de arquitectura

Figura 1. Vista de integración: frontend React, API Gateway, Eureka, microservicios, bases MySQL y OpenCage.

2.1 Vista lógica (Mermaid)

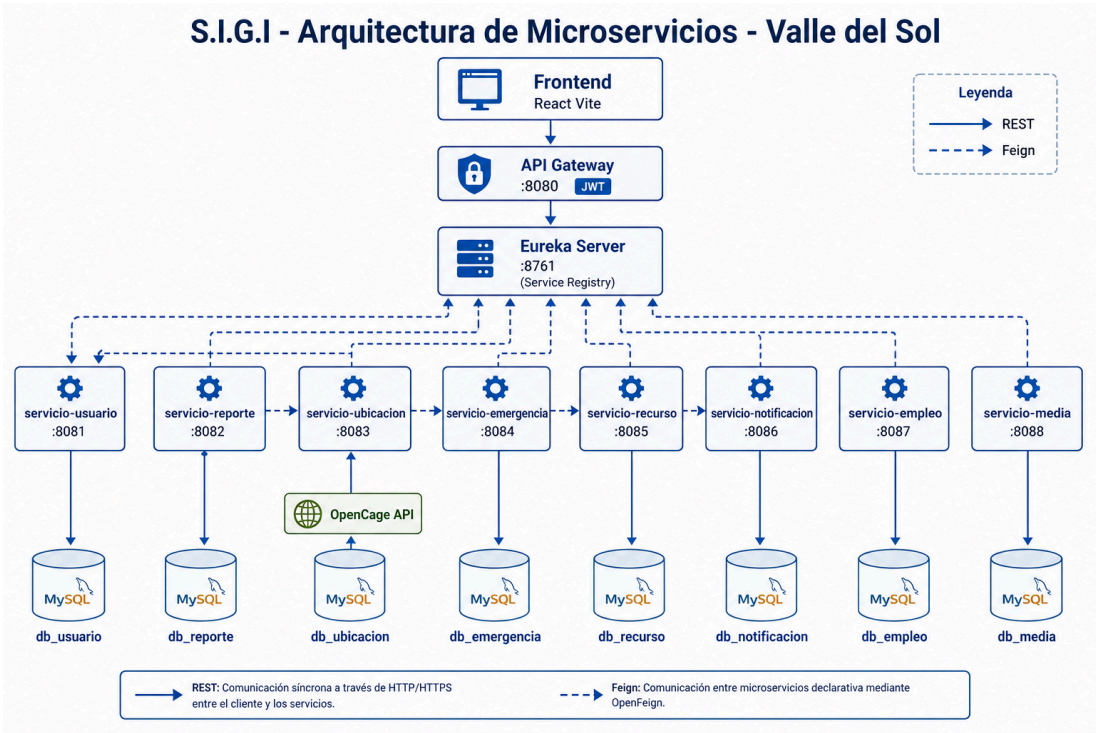


Diagrama de arquitectura

3. Componentes y tecnologías

Componente	Tecnología	Puerto	Función
Frontend	React 19, Vite, Tailwind	5173	UI ciudadana y operadores
API Gateway	Spring Cloud Gateway, jjwt	8080	Punto único de entrada, validación JWT
Eureka	Netflix Eureka Server	8761	Registro y descubrimiento
Microservicios	Spring Boot 3.5.14	8081–8088	Lógica de negocio + JPA
MySQL	MySQL 8.0 (Docker)	13306 host	Una base lógica por servicio
OpenCage	API REST externa	—	Geocodificación de direcciones

3.1 Bibliotecas principales del backend

Biblioteca	Uso	Por qué la elegimos
Spring Boot 3.5	Base de cada microservicio	Productividad, autoconfiguración, ecosistema maduro
Spring Cloud 2025.0	Gateway, Eureka, OpenFeign	Integración probada entre servicios
Spring Data JPA	Persistencia	CRUD automático, @Entity , repositorios
Hibernate	ORM detrás de JPA	ddl-auto=update en desarrollo
MySQL Connector/J	Driver JDBC	Compatible con MySQL 8 del docker-compose
jjwt 0.11.5	Tokens JWT	Mismo secret en gateway y servicio-usuario
Resilience4j	Circuit breaker	Fallback si servicio-ubicacion no responde
springdoc-openapi	Swagger UI	Documentación viva de cada API
Lombok	DTOs y entidades	Menos boilerplate en el código
BCrypt	Hash de contraseñas	Seguridad en servicio-usuario

4. Comunicación entre servicios

4.1 Cliente → Backend

El frontend **solo** habla con `http://localhost:8080` . El Gateway enruta por path:

- `/auth/**` → servicio-usuario (sin JWT en registro/login)
- `/api/**` → microservicio correspondiente (con filtro JwtAuth)

Tras validar el token, el Gateway inyecta cabeceras:

- `X-User-Id`
- `X-User-Name`
- `X-User-Role`

Los controladores confían en esas cabeceras para autorización por rol.

4.2 Backend → Backend (Feign y REST)

Origen	Destino	Mecanismo	Ejemplo
servicio-reporte	servicio-ubicacion	OpenFeign + Circuit Breaker	Coordenadas de una dirección
servicio-reporte	servicio-emergencia	OpenFeign	Crear emergencia al validar reporte
servicio-emergencia	servicio-recurso	OpenFeign	Asignar camiones/brigadas
servicio-emergencia	servicio-notificacion	OpenFeign	Alertas a vecinos y reportante
servicio-empleo	servicio-usuario	RestTemplate	Datos del postulante

Eureka resuelve los nombres lógicos (`lb://servicio-reporte`) dentro de Docker.

5. Patrones que aplicamos

1. **Database per service:** cada microservicio tiene su schema (`db_reporte` , `db_emergencia` , etc.) creado en `docker/mysql-init/01-databases.sql`.
2. **API Gateway / BFF:** un solo punto público; los microservicios no se exponen al navegador.
3. **Repository (Spring Data):** interfaces `JpaRepository` por entidad.
4. **DTO:** separación entre entidad JPA y JSON de la API (`ReporteResponse` , `UsuarioResponseDTO` , etc.).
5. **Circuit Breaker:** en `UbicacionConsultaService` para no bloquear la creación de reportes.
6. **Singleton (Spring):** beans `@Service` con una instancia por contexto.

6. Seguridad en la arquitectura

- **JWT** firmado con HS256; expiración 24 h.
- **BCrypt** strength 12 para contraseñas.
- Gateway rechaza peticiones sin token en rutas `/api/**` (excepto rutas públicas configuradas).
- Cada microservicio tiene `SecurityConfig` con `permitAll()` porque la autenticación real ocurre en el Gateway (modelo demo académico).

7. Despliegue

- **Desarrollo:** `docker compose up --build` levanta MySQL, Eureka, Gateway y los ocho servicios.
 - **Kubernetes:** manifiestos de ejemplo en `k8s/` para demo con una réplica.
 - **Pruebas unitarias:** Maven local sin Docker (`mvn verify`).
-